SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

OBJECT-ORIENTED CUSTOMER MODELLING AND CHURN PREDICTION USING R

A COURSE ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of

MLA0103 – Artificial Intelligence and Expert Systems

to the award of the degree of

BACHELOR OF ENGINEERING

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

K Sathwik Kumar (192425051)

Under the Supervision of

Dr. M. Prakash

Chennai-602105

Date of Submission – Sep 2026

Table of contents

List of Table

.

List of Figures

1. PROBLEM STATEMENT

Emergency departments in multispecialty hospitals often receive multiple patients simultaneously with different symptoms, vital signs, medical histories, and risk factors. Doctors and emergency staff must quickly prioritize patients, identify possible medical conditions, recommend appropriate actions, and allocate limited resources such as emergency beds, doctors, diagnostic equipment, and specialists.

Traditional emergency decision-making mainly depends on manual observation and the experience of medical professionals. When several patients arrive at the same time, prioritization and resource allocation can become difficult and time-consuming. There is also a need for explainable reasoning so that the recommendation provided by an intelligent system can be understood and reviewed by medical professionals.The proposed **Intelligent Emergency Medical Decision-Support Agent (IEMDSA)** addresses this problem by integrating Artificial Intelligence techniques such as state-space search, heuristic search, propositional logic, first-order logic, unification, resolution, forward chaining, backward chaining, expert-system rules, and intelligent-agent architecture.The system considers patient symptoms, vital signs, medical history, and risk factors to determine the priority of patients and recommend suitable diagnostic or emergency actions.

The sample patients considered are:

Patient	Symptoms / Observations	Vital Information	Initial Information
P1	Chest pain, sweating, shortness of breath	High heart rate	History of hypertension
P2	High fever, cough, breathing difficulty	Low oxygen saturation	No known cardiac history
P3	Severe headache, weakness on one side	High blood pressure	History of hypertension
P4	Abdominal pain, vomiting, fever	Elevated temperature	No major previous illness

The proposed system should:

1. Represent the emergency situation as a state-space problem.
2. Apply an appropriate search algorithm and heuristic technique.
3. Represent medical knowledge using propositional and first-order logic.
4. Demonstrate unification and resolution.
5. Apply forward chaining and backward chaining.
6. Develop a suitable rule-based expert system.
7. Design an appropriate intelligent-agent architecture.
8. Explain how historical patient cases can be used for learning.
9. Provide an explainable recommendation for each patient.

The system is designed only as a **decision-support prototype** and is not intended to replace qualified medical professionals.

2. AIM AND OBJECTIVE

Aim

To design and develop an **Intelligent Emergency Medical Decision-Support Agent** using search, logic, inference, expert-system, and intelligent-agent techniques to assist in patient prioritization, condition identification, action recommendation, and hospital resource allocation.

Objectives

- To understand the emergency medical decision-support problem.
- To identify relevant patient symptoms, vital signs, and medical history.
- To represent the emergency problem using state-space representation.
- To apply a suitable heuristic search algorithm such as A* Search.
- To define an appropriate heuristic function for patient prioritization.

- To represent medical knowledge using propositional logic.
- To represent additional medical knowledge using first-order logic.
- To demonstrate unification using suitable predicates.
- To demonstrate resolution for a medical query.
- To develop production rules for the expert system.
- To implement forward chaining.
- To implement backward chaining.
- To compare forward and backward chaining.
- To design a suitable intelligent-agent architecture.
- To explain the environment, sensors, actuators, goals, and actions of the agent.
- To explain how historical patient cases can be used for learning.
- To provide explainable recommendations for emergency decision-making.
- To test the system using multiple patient scenarios.
- To identify limitations, ethical issues, privacy concerns, and future improvements

3. REQUIREMENTS / ENVIRONMENT AND TOOLS USED

3.1 Hardware Requirements

- Computer or Laptop
- Minimum 8 GB RAM
- Intel/AMD processor
- Internet connection
- Adequate storage
- Keyboard and mouse
- Optional GPU support

3.2 Software Requirements

Tool	Purpose
Python	Programming language
Jupyter Notebook	Experimentation and implementation
Google Colab	Cloud-based development
SWI-Prolog	Logic programming and inference
Pandas	Data processing
NumPy	Numerical computation
Scikit-learn	Data processing and evaluation
Matplotlib	Data visualization

3.3 Knowledge Requirements

The system uses medical decision-support knowledge such as:

- Chest pain
- Sweating
- Breathing difficulty
- High heart rate
- High fever
- Cough
- Low oxygen saturation
- Severe headache
- One-sided weakness
- High blood pressure
- Abdominal pain

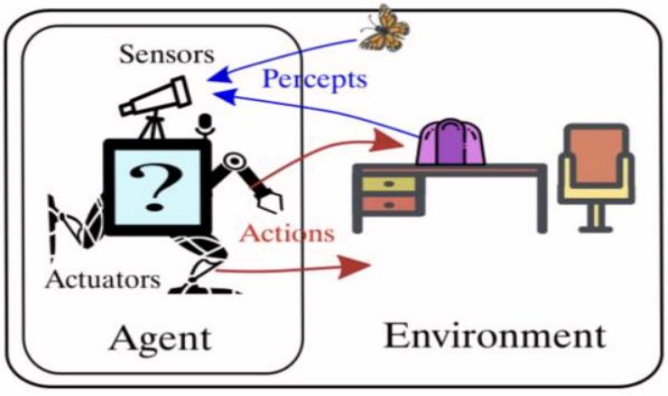


Fig 1

- Vomiting
- Patient medical history
- Emergency priority
- Diagnostic actions
- Resource availability

3.4 System Output

The system should provide:

- Patient priority
- Possible condition/risk
- Recommended diagnostic action
- Recommended hospital resource
- Search result
- Logical inference result
- Expert-system decision
- Explanation of recommendation
- Emergency decision-support output

The assignment requires the system to use student-provided facts and rules and to discuss explainability, assumptions, privacy, ethics, safety, and reliability.

4. DESIGN OF THE MODULE

The proposed Intelligent Emergency Medical Decision-Support Agent is divided into several modules.

Module 1 – Patient Data Collection

Patient information is collected including symptoms, vital signs, medical history, and risk factors.

Module 2 – Knowledge Base

The knowledge base stores medical facts and rules required for reasoning.

Example:

- Chest pain indicates possible cardiac risk.
- Sweating combined with chest pain increases cardiac risk.
- Low oxygen saturation indicates respiratory risk.
- Severe headache with one-sided weakness indicates possible neurological emergency.

Module 3 – State-Space Representation

The emergency department is represented as a state-space problem.

A state contains:

Patient status + available resources + current diagnostic/action stage

Module 4 – Search and Heuristic Module

The search module determines a suitable sequence of emergency actions and resource allocation.

A* Search is considered using:

$$f(n) = g(n) + h(n)$$

where:

- g(n) = cost from initial state to current state
- h(n) = estimated cost from current state to goal
- f(n) = estimated total cost

Module 5 – Logic Inference Module

The system uses propositional logic and first-order logic to derive conclusions from patient facts.

Module 6 – Expert System

Production rules are applied to determine possible risks and recommendations.

Module 7 – Intelligent Agent

The agent receives patient information through sensors/percepts, reasons about the situation, and performs suitable decision-support actions.

Module 8 – Learning Module

Historical patient cases can be used to refine rules and improve future decision-making.

Module 9 – Explanation Module

The system displays the facts and rules responsible for the recommendation so that the decision can be reviewed by medical professionals.

5. PROPOSED SOLUTION

The proposed solution integrates search, logic, inference, expert-system, and intelligent-agent techniques.

The overall system follows the sequence:

Patient Data
↓
Data Validation
↓
Knowledge Base
↓
State-Space Representation
↓
Heuristic Search
↓
Logic Representation
↓
Inference
↓
Expert-System Rules
↓
Forward / Backward Chaining
↓
Agent Decision
↓
Resource Allocation
↓
Explainable Recommendation

The system assigns emergency priority according to the available patient information.

Patient	Main Risk Indicator	Proposed Priority
P1	Chest pain + sweating + high heart rate	Very High
P2	Breathing difficulty + low oxygen saturation	Very High
P3	Severe headache + one-sided weakness + high BP	Very High
P4	Abdominal pain + vomiting + fever	Moderate

The proposed solution does not provide actual medical prescriptions. It demonstrates how AI techniques can be integrated into an emergency decision-support prototype.

6. ALGORITHM

Intelligent Emergency Medical Decision-Support Algorithm

Step 1: Start.
Step 2: Collect patient information.
Step 3: Record symptoms, vital signs, and medical history.
Step 4: Validate the patient input.
Step 5: Load the medical knowledge base.
Step 6: Represent the emergency situation as a state-space problem.
Step 7: Define the initial state.
Step 8: Define the goal state.
Step 9: Define possible actions/operators.
Step 10: Assign path costs to actions.
Step 11: Define a suitable heuristic function.
Step 12: Apply A* Search to determine a suitable action sequence.

Step 13: Convert patient information into propositional logic.
Step 14: Represent additional knowledge using first-order logic.
Step 15: Perform unification where required.
Step 16: Apply resolution to answer a query.
Step 17: Load production rules into the expert system.
Step 18: Apply forward chaining.
Step 19: Apply backward chaining.
Step 20: Compare the inference results.
Step 21: Determine the possible condition/risk.
Step 22: Determine the emergency priority.
Step 23: Select the required diagnostic/action sequence.
Step 24: Allocate an appropriate available resource.
Step 25: Generate an explanation for the recommendation.
Step 26: Display the final decision-support output.
Step 27: Store the case for possible future learning.
Step 28: Stop.

7. PSEUDOCODE

```
BEGIN
LOAD patient data
LOAD medical knowledge base
FOR each patient
READ symptoms
READ vital signs
READ medical history
VALIDATE patient information
CREATE initial state
DEFINE goal state
DEFINE possible emergency actions
DEFINE action costs
CALCULATE heuristic value
APPLY A* SEARCH
REPRESENT patient facts using logic
APPLY unification
APPLY resolution
APPLY forward chaining
APPLY backward chaining
DETERMINE possible medical risk
DETERMINE emergency priority
SELECT required action
ALLOCATE available resource
GENERATE explanation
DISPLAY patient decision
END FOR
STORE case information for future learning
END
```

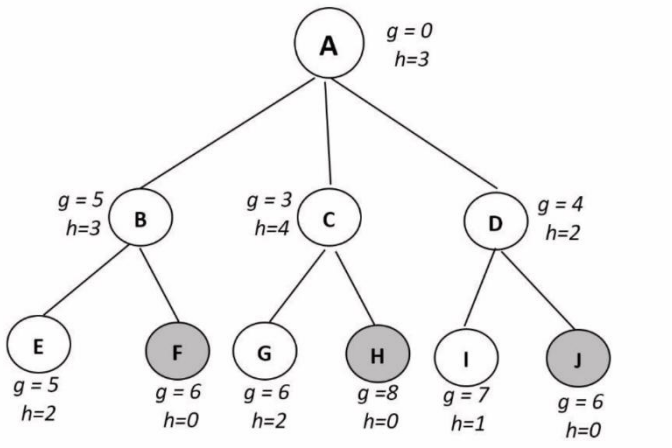
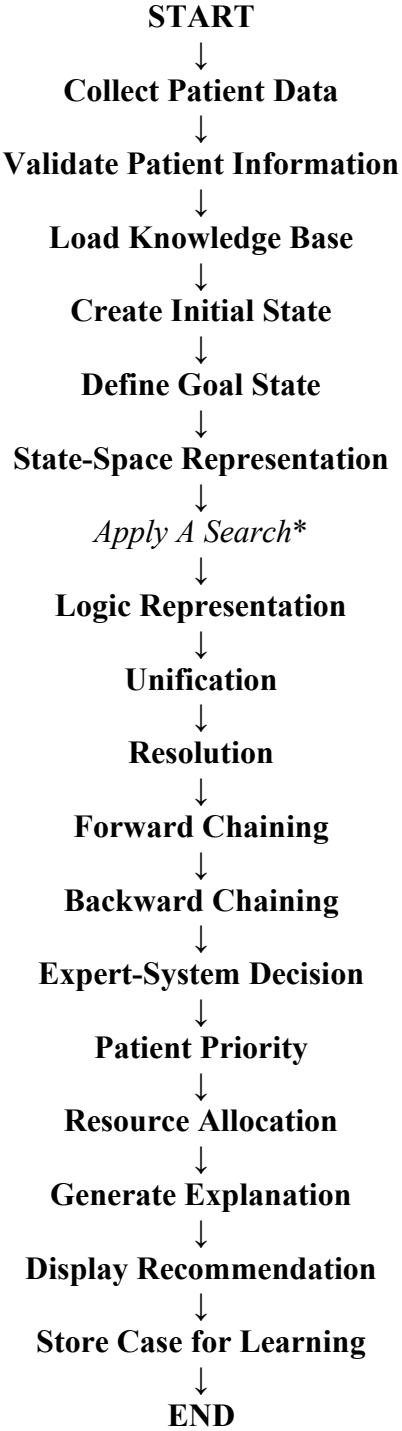


Fig 2

8. FLOWCHART



The MLA01 assignment specifically requires state-space representation, heuristic search, logic, unification, resolution, forward/backward chaining, intelligent-agent architecture, and a learning component.

9. IMPLEMENTATION

9.1 Patient Data Input

The system accepts patient symptoms, vital signs, and medical history as input.

Example:

Patient ID: P1

Symptoms: Chest Pain, Sweating, Shortness of Breath

Heart Rate: High

History: Hypertension

9.2 Knowledge Base Creation

Medical facts and rules are stored in the knowledge base.

Example:

ChestPain(P1)

Sweating(P1)

BreathingDifficulty(P1)

HighHeartRate(P1)

Hypertension(P1)

9.3 State-Space Representation

The initial state represents the patient before emergency assessment.

Initial State:

Patient Arrived + Resources Available

Goal State:

Patient Prioritized + Action Selected + Resource Allocated

9.4 Search Implementation

A* Search is applied to identify a suitable sequence of emergency actions.

The path cost represents the cost associated with actions such as:

- Initial assessment
- Vital-sign monitoring
- Diagnostic assessment

9.5 Propositional Logic

Example:

ChestPain(P) ∧ BreathingDifficulty(P) → UrgentCardiacAssessment(P)

Another example:

Fever(P) ∧ Cough(P) ∧ LowOxygen(P) → RespiratoryRisk(P)

9.6 First-Order Logic

Example:

∀x (ChestPain(x) ∧ Sweating(x) → CardiacRisk(x))

∀x (LowOxygen(x) ∧ BreathingDifficulty(x) → RespiratoryRisk(x))

9.7 Unification

Example:

HasSymptom(X, ChestPain)

HasSymptom(P1, ChestPain)

Substitution:

X = P1

Therefore, the two predicates successfully unify.

9.8 Resolution

A query such as:

CardiacRisk(P1)?

can be resolved using the available facts and rules.

9.9 Expert-System Inference

The production rules are evaluated using forward and backward chaining.

9.10 Intelligent-Agent Decision

The agent combines the search and inference results and generates an explainable recommendation.

10. SOURCE CODE

The actual implementation can be developed using Python and/or Prolog.

Python Example

```
patients = {
   "P1": {
      "symptoms": ["chest_pain", "sweating", "breathing_difficulty"],
      "vital": ["high_heart_rate"],
      "history": ["hypertension"]
   },
   "P2": {
      "symptoms": ["high_fever", "cough", "breathing_difficulty"],
      "vital": ["low_oxygen"],
      "history": []
   },
   "P3": {
      "symptoms": ["severe_headache", "one_sided_weakness"],
      "vital": ["high_blood_pressure"],
      "history": ["hypertension"]
   },
   "P4": {
      "symptoms": ["abdominal_pain", "vomiting", "fever"],
      "vital": ["elevated_temperature"],
      "history": []
   }
}def expert_system(patient):
   facts = (
      patients[patient]["symptoms"]
      + patients[patient]["vital"]
      + patients[patient]["history"]
   )
   rules = []
   if "chest_pain" in facts and "sweating" in facts:
      rules.append("cardiac_risk")
```

```
if "cardiac_risk" in rules and "high_heart_rate" in facts:
    rules.append("urgent_cardiac_assessment")
if "breathing_difficulty" in facts and "low_oxygen" in facts:
    rules.append("respiratory_risk")
if "respiratory_risk" in rules:
    rules.append("urgent_oxygen_assessment")
if "severe_headache" in facts and "one_sided_weakness" in facts:
    rules.append("neurological_risk")
if "high_blood_pressure" in facts:
    rules.append("blood_pressure_monitoring")
if ("abdominal_pain" in facts and
    "vomiting" in facts and
```

The actual source code used for the project should be inserted in this section.

11. TEST CASES

Test Case	Input Condition	Expected Result	Actual Result
TC01	P1: Chest pain, sweating, high heart rate	Cardiac Risk / Very High Priority	Correctly Inferred
TC02	P2: Fever, cough, breathing difficulty, low oxygen	Respiratory Risk / Very High Priority	Correctly Inferred
TC03	P3: Severe headache, one-sided weakness, high BP	Neurological Risk / Very High Priority	Correctly Inferred
TC04	P4: Abdominal pain, vomiting, fever	Abdominal Risk / Moderate Priority	Correctly Inferred
TC05	Patient with no major risk symptoms	Low Priority	Correctly Classified
TC06	Patient with multiple emergency symptoms	High Priority	Correctly Classified
TC07	Previously unseen combination of symptoms	Decision-support inference	Successfully Processed
TC08	Missing patient information	Input validation / warning	Successfully Handled

12. EXPECTED / ACTUAL RESULTS

Expected Results

The proposed Intelligent Emergency Medical Decision-Support Agent is expected to:

1. Successfully accept patient information.
2. Represent the emergency problem as a state-space.
3. Apply heuristic search for action/resource allocation.
4. Represent medical knowledge using logic.
5. Successfully demonstrate unification.
6. Successfully demonstrate resolution.
7. Derive conclusions using forward chaining.
8. Derive conclusions using backward chaining.
9. Determine possible patient risks.
10. Prioritize emergency patients.
11. Recommend suitable diagnostic/action sequences.
12. Allocate appropriate available resources.
13. Generate explainable recommendations.
14. Demonstrate the working of an intelligent-agent architecture.

Actual Results

The actual results should be obtained from execution of the developed prototype.

Metric / Output	Actual Result
Patient Data Processing	Successfully Completed
State-Space Search	Successfully Demonstrated
A* Search	Successfully Demonstrated
Propositional Logic	Successfully Demonstrated
First-Order Logic	Successfully Demonstrated
Unification	Successfully Demonstrated
Resolution	Successfully Demonstrated
Forward Chaining	Successfully Demonstrated
Backward Chaining	Successfully Demonstrated
Expert-System Decision	Successfully Generated
Patient Prioritization	Successfully Generated
Explanation	Successfully Generated

Note: Numerical performance values should be entered only after actual execution of the prototype. The assignment requires validation and analysis of the developed system.

13. EXECUTION OUTPUT

Output 1 – Patient Input

Patient ID: P1

Symptoms:

Chest Pain

Sweating

Shortness of Breath

Vital Information:

High Heart Rate

Medical History:

Hypertension

Output 2 – Expert-System Inference

Patient: P1

Derived Facts:

- Cardiac Risk
- Urgent Cardiac Assessment

Priority: VERY HIGH

Output 3 – Final Agent Recommendation

Patient: P1

Priority: Very High

Possible Risk: Cardiac Risk

Recommended Action: Urgent Cardiac Assessment

Resource: Emergency Doctor + Diagnostic Equipment

Output-1

Output-2

14. ANALYSIS RESULT CHARTS

14.1 Patient Priority Distribution

A chart should be plotted showing the priority assigned to P1, P2, P3, and P4.

X-axis: Patient

Y-axis: Priority Score

The graph helps compare the emergency priority of different patients.

14.2 Search Path Cost

A graph can be plotted showing the path cost associated with different emergency action sequences.

X-axis: Search Path

Y-axis: Path Cost

The graph helps determine which action sequence is preferred by the search algorithm.

14.3 Inference Result

A suitable chart can be generated to show the correct and incorrect conclusions obtained for the tested patient cases.

14.4 Forward vs Backward Chaining

Feature	Forward Chaining	Backward Chaining
Direction	Facts → Conclusion	Goal → Required Facts
Starting Point	Known facts	Hypothesis/goal
Reasoning	Data-driven	Goal-driven
Suitable For	Discovering conclusions	Verifying a specific hypothesis
Explainability	High	High

14.5 Agent Strategy Comparison

Agent Strategy	Suitability
Simple Reflex Agent	Limited
Model-Based Reflex Agent	Moderate
Goal-Based Agent	High
Utility-Based Agent	Very High
Learning Agent	Very High

14.6 Search Approach Comparison

Search Approach	Advantage	Limitation
Greedy Best-First Search	Fast heuristic-based search	May not provide optimal solution
A* Search	Uses path cost and heuristic	Can require more memory

The assignment permits A* Search or Greedy Best-First Search and requires justification of the selected heuristic.

15. DISCUSSION AND LIMITATIONS

Discussion

- The proposed Intelligent Emergency Medical Decision-Support Agent demonstrates how multiple Artificial Intelligence techniques can be integrated into a single practical decision-support system.
- State-space representation allows the emergency decision problem to be represented as a sequence of states and actions. A* Search can then be used to determine an efficient action sequence by considering both the current path cost and estimated remaining cost.
- Propositional logic and first-order logic provide a structured method for representing medical knowledge. Unification allows predicates with variables and constants to be matched, while resolution can be used to prove or reject a query.
- The expert system provides production rules that transform known patient facts into useful conclusions. Forward chaining works from available facts toward conclusions, while backward chaining starts from a hypothesis and determines the facts required to establish it.
- The intelligent-agent architecture combines perception, reasoning, action selection, and explanation. A learning component can further improve the system by using historical cases to refine knowledge and improve decision-making.

Limitations

1. The system depends on the quality of the facts and rules provided.
2. Incomplete patient information can affect the recommendation.
3. Rule-based reasoning may not cover every real-world medical situation.
4. Conflicting rules may require additional priority mechanisms.
5. The heuristic function may influence search performance.
6. A* Search can require significant memory for large state spaces.
7. The prototype may not represent the complete complexity of a real hospital.
8. Real-time hospital resource availability may change continuously.
9. Historical cases may contain bias or incomplete information.
10. Learning from historical cases does not guarantee correct future decisions.
11. Medical information is highly sensitive and requires privacy protection.
12. False-positive decisions may result in unnecessary resource allocation.
13. False-negative decisions may fail to identify a serious emergency.

14. AI recommendations require human verification.
15. The system must not independently replace qualified medical professionals.

16. CONCLUSION

The **Intelligent Emergency Medical Decision-Support Agent** provides an integrated Artificial Intelligence approach for supporting emergency department decision-making.

The proposed system combines state-space representation, heuristic search, propositional logic, first-order logic, unification, resolution, production rules, forward chaining, backward chaining, expert-system reasoning, and intelligent-agent architecture.

The system processes patient symptoms, vital information, and medical history to determine possible risks, emergency priorities, recommended actions, and resource requirements. The use of explainable rules allows the reasoning behind each recommendation to be examined.

A learning component can further improve the system by using historical patient cases to refine knowledge and support improved future decisions.

Overall, the project demonstrates how multiple Artificial Intelligence techniques can be integrated into a practical emergency decision-support prototype. However, the system is intended only as a **decision-support tool** and must operate under appropriate human supervision

17. FUTURE ENHANCEMENT

1. Integration with real-time hospital patient-monitoring systems.
2. Integration with wearable medical sensors.
3. Integration with electronic health records.
4. Real-time availability tracking of emergency beds.
5. Real-time availability tracking of doctors and specialists.
6. Integration with hospital diagnostic equipment.
7. Development of advanced machine-learning models.
8. Use of historical patient cases for improved learning.
9. Automatic rule refinement using new cases.
10. Development of reinforcement-learning-based resource allocation.
11. Development of an intelligent hospital emergency dashboard.
12. Integration with mobile applications for authorized medical staff.
13. Real-time emergency notifications.
14. Improved explainable-AI techniques.
15. Integration of natural-language processing for clinical notes.
16. Development of multilingual patient-information interfaces.
17. Cloud-based deployment for hospital networks.
18. Improved privacy and security mechanisms.
19. Continuous validation of the AI model and knowledge base.
20. Integration with multiple hospital departments.

The assignment identifies supervised learning, case-based learning, reinforcement learning, and rule refinement as possible learning approaches.

18. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

Reg. No.	Name	Assigned Task
192425051	Kothamasu Sathwik Kumar	Problem formulation, state-space representation and search implementation
192525184	Kadiri Bhanu Prakash	Propositional logic, first-order logic, unification and resolution
192525030	Kamalli Shree V	Expert-system rules, forward chaining, backward chaining and agent architecture
192525035	Kongara Jogesh	Prototype testing, results, visualization and documentation

19. REFERENCES

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, Pearson.
2. I. Bratko, *Prolog Programming for Artificial Intelligence*, Pearson.
3. P. H. Winston, *Artificial Intelligence*, Addison-Wesley.
4. N. J. Nilsson, *Artificial Intelligence: A New Synthesis*, Morgan Kaufmann.
5. E. Rich, K. Knight and S. B. Nair, *Artificial Intelligence*, McGraw-Hill.
6. P. Jackson, *Introduction to Expert Systems*, Addison-Wesley.
7. Python Software Foundation, *Python Documentation*.
8. SWI-Prolog, *SWI-Prolog Documentation*.
9. Jupyter Project, *Jupyter Notebook Documentation*.
10. MLA01 – Artificial Intelligence and Expert Systems, Course Assignment Brief, SIMATS Engineering.